

Labwork 6: Map

Trinh Thanh Vinh

October 3, 2026

1 Introduction

This labwork is about using CUDA map pattern for image processing. The same function is applied to every pixel of the image. All the pixels are processed in parallel. Three programs were implemented: grayscale binarization, brightness control, and image blending.

2 Implementation

First, image is loaded as an 8-bit value array. The array is then copied to the GPU, and a 2D CUDA grid is created. Each thread calculated its pixel position (x, y) and checked that the position was inside the image. The grid size is also calculated to make sure the program works for any image size.

2.1 Grayscale Binarization

For every pixel, the kernel calculated the gray value from the average of the red, green, and blue channels. It then compared this value with a threshold. If the value is smaller than the threshold, the pixel is set to 0 (black); otherwise, it is set to 1 (white). The threshold can be adjusted to achieve different results, and it must be in the range of 0 to 255. The result is clamped to the range of 0 to 1.



Figure 1: Binarization results with three block sizes.

2.2 Brightness Control

The kernel added a positive or negative value, ranged from -255 to 255, to every RGB channels. Negative values darken the image, while positive values brighten it. The result is clamped to the range of 0 to 255 to avoid overflow or underflow.

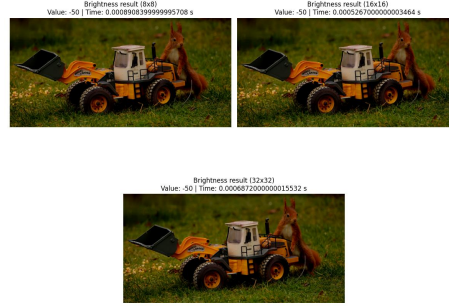


Figure 2: Brightness results with three block sizes.

2.3 Image Blending

Two images with the same size are the input for this program. This is achieved by calculating a weighted average for each color channel. The weight is a value between 0 and 1, which can be adjusted to achieve different results. The result is clamped to the range of 0 to 255.



Figure 3: Blending results with three block sizes.

3 Results

The block sizes 8×8 , 16×16 , and 32×32 were tested. Before measuring, the kernel ran once as a warm-up. Each reported time is the average of five kernel runs. Data transfer time was not included.

Table 1: Average GPU kernel time in milliseconds.

Operation	8×8	16×16	32×32
Binarization	2.754	2.705	2.779
Brightness	0.891	0.527	0.687
Blending	4.779	4.647	5.283

The 16×16 block size was the fastest in all three tests. It gave a good balance between the number of threads in a block and the number of blocks. The differences were small, so the exact times can change with the GPU and the current computer load.